第17天 · 先审阅，再执行写入

让批准绑定到具体内容，用幂等记录避免重复执行。

标准路线：60分钟；忙碌日：20分钟；扩展练习另计。

今天的目标

- 复现 needs_approval、written 与 already_applied 三种状态。

- 验证改变草稿后旧批准失效。

- 解释内容摘要、身份授权与科学复核的不同作用。

开始之前

- 已经完成第 16 课，或能解释其产物。

- 打开实践包 practice 目录；后文 python 可替换为虚拟环境内解释器的完整路径。

一小时安排

- 15分钟 · 理解概念：阅读本课四节讲解，弄清一个关键机制与适用边界。

- 30分钟 · 亲手操作：先跑基线，再做一次单变量实验，保留前后结果。

- 10分钟 · 检查结果：逐项验收，完成两道自测并阅读解析。

- 5分钟 · 留下记录：写出本课发现、未解决问题和下次第一步。

01 · 理解概念

批准的是具体动作与具体内容

“可以写报告”仍不够具体。一次可检查的批准应当说明写在哪里、准备写什么，以及执行会改变什么。本课固定写入项目输出目录中的 approved_report.txt，避免把演示扩展成任意路径写入工具。

第一次运行时不要填批准值。先阅读 preview 和 digest；程序应停在 needs_approval。只有你看过将写入的内容，才把相应摘要复制到 approved_digest 中，再运行同一任务。

摘要把批准和内容联系起来

本课摘要由固定目标路径与待写内容共同计算。相同目标和内容得到相同摘要；内容改变后，之前的批准不应继续适用。这能减少“审阅的是旧稿，执行的是新稿”的错误。

摘要不是加密，也不是完整的权限系统。本课的摘要用于把一次演示批准与文件内容联系起来。真实产品还需要验证批准者身份、权限、有效期与操作目标。

幂等让重复请求不重复做事

网络中断后用户可能再点一次提交。幂等设计让同一批准、同一内容重复提交时，报告 already_applied，而不是重复追加或制造多个副本。这里判断的是相同操作是否已经执行。

幂等不等于所有状态永远不变。新内容应产生新的摘要与新的审阅要求；不同动作还需要不同的操作标识。把全局“已批准”布尔值当作永久授权，会失去这层边界。

在科研流程中使用审阅节点

你可以把写入前预览迁移到分析报告、候选文献清单或批量文件整理。自动化负责准备结果，人负责确认具体内容，再让程序执行有限操作。

本课只写教学报告，不提交论文、不发送邮件，也不修改原始科研数据。完成后要能指向输出文件和批准记录，而不能把“程序生成了一份文本”当成内容已经经过科学审核。

02 · 一步一做

命令在解压后的 practice 文件夹执行。Windows可使用 .\.venv\Scripts\python.exe；macOS/Linux可使用 .venv/bin/python 替代 python。

1. 先查看草稿，保持未批准状态

为什么做：先知道程序读什么，后面才能解释输出为何改变。

确认 day17.approved_digest 为空字符串，再运行第17课。检查 preview 是否是你愿意写入的内容。

```
python run.py day 17
```

预期结果：status 为 needs_approval，输出待写预览与 digest。

2. 批准当前草稿后执行

为什么做：保持默认设置的一次运行，是后续比较的参照。

在输出中复制完整 digest，粘贴到 experiments.json 的 day17.approved_digest；不要改 draft_text，保存后再运行。

```
python run.py day 17
```

预期结果：status 为 written，固定输出文件包含刚才审阅过的内容。

3. 重复请求，检查幂等

为什么做：不能只看程序有没有报错，要检查它实际做了什么。

保持草稿和批准摘要完全不变，再次运行并查看结果。

```
python run.py day 17
python -m json.tool outputs/day17.json
```

预期结果：status 为 already_applied，重复执行没有再次追加内容。

4. 改变一个条件并比较

为什么做：主动制造可解释的变化，才能验证自己理解的机制。

完成下面“改一改”实验。对比基线与新结果，记录哪些字段改变、哪些保持稳定；不要同时改模型、材料和参数。

预期结果：留下一组前后结果和一句解释；失败状态也可以是正确的实验产物。

改一改 · 修改草稿，观察旧批准为什么失效

- 先执行 python run.py day 17，将 outputs/day17.json 复制为自己的基线备份。

- 在 VS Code 打开 experiments.json，只修改 day17.draft_text 为 "Teaching report: this is a revised draft."，保持其他键不变并保存。

- 执行 python -m json.tool experiments.json 检查语法，再执行 python run.py day 17。

- 对比两份结果，重点观察status、digest、preview 和 path；解释改变原因后恢复原设置或明确记录保留值。

观察什么：先完成一次批准写入，再只修改 draft_text 并保留旧 approved_digest；程序应拒绝把旧批准用于新内容。

机制解释：审批必须绑定到当前具体内容，不能因为用户曾经批准过一次就持续执行变化后的动作。

常见问题

修改配置后结果没有变化

原因：改了其他天的字段，或编辑后尚未保存。

处理：核对 experiments.json 中的 day17，保存后重新运行；检查实际执行目录。

把受控失败误当成程序坏了

原因：本课实验主动制造了边界条件。

处理：先对照 expected 和状态字段，区分设计内拒绝与未处理异常；保留错误信息再修复。

03 · 验收

让批准绑定到具体内容，用幂等记录避免重复执行。 保留基线、修改结果与自己的解释。

成果文件：outputs/day17.json

☐ 未批准时不发生正式写入。

☐ 批准后写入成功，重复执行得到 already_applied。

☐ 修改草稿后旧摘要被拒绝，已记录审阅边界。

自测与解析

1. 修改待写内容后，之前的 approved_digest 应如何处理?

- A. 永久有效

- B. 自动替换为新摘要

- C. 需要重新审阅并批准

- D. 忽略摘要继续执行

答案：C。 旧批准只对应旧内容，新内容需要新的审阅。

2. already_applied 主要说明什么?

- A. 科学结论已被验证

- B. 相同操作已执行，重复请求没有再次写入

- C. 所有用户都拥有写权限

- D. 报告内容永远不能改

答案：B。 这是执行层面的幂等状态，不代表科学正确性或普遍授权。

小词典

- Digest：由内容计算的摘要，可用于检验内容是否改变。

- Idempotency：重复执行同一操作时避免重复产生效果的性质。

- Approval：用户对明确对象、内容与动作作出的批准。

04 · 我的记录

- 我的科研流程里哪一步值得在写入前显示预览？

- 如果批准后文件内容又改变，应该由哪一层阻止？

忙碌日：20分钟

- 用5分钟阅读第一个概念，写出本课要解决的问题。

- 用10分钟运行 python run.py day 17，打开 outputs/day17.json，检查一个关键字段。

- 用5分钟完成第一道自测并留一句解释，完整实验与验收留到下次。

保存一份运行结果和自己的解释；精简练习单独记录，不等于整课验收全部完成。

有余力再做：20分钟选学 · 把本课机制迁移到一个自己的任务

- 从自己的文献整理或数据分析流程中选一个相似环节。

- 写清输入、允许动作、失败出口与验收方法。

- 提出一个会让当前实现失败的反例，并标明它是待实现扩展。

参考资料与阅读范围

Python：hashlib：阅读 hashlib.sha256 的输入、digest 与 hexdigest。

https://docs.python.org/3/library/hashlib.html

MCP：2025-11-25 协议规范：阅读 Security and Trust & Safety 中的用户控制与工具安全。

https://modelcontextprotocol.io/specification/2025-11-25

下一步

下一课把工具连接放进真实的 stdio 消息往返中。